

A FREE GUIDE · MWONGOZO WA BURE

# Python Daily.

From beginner to professional

*Kidogo kidogo, kila siku*

Six levels, real code, practice and projects, and every area where Python is used.



---

**Privatus Cosmas**

depriver.tech · @\_depriver

# Python Daily

From beginner to professional

*Kidogo kidogo, kila siku*

---

Privatus Cosmas

## **Python Daily: From beginner to professional**

© 2026 Privatus Cosmas. First edition, October 2026.

Free to read, print and share with anyone learning. Please don't sell it.

Python is a trademark of the Python Software Foundation. Other names belong to their owners.

Code examples use Python 3.13. Tools and versions change; the ideas don't.

**Latest version and the companion carousel: [depriver.tech/blog/python-roadmap](https://depriver.tech/blog/python-roadmap)**

# What's inside.

|                |                                            |    |
|----------------|--------------------------------------------|----|
| <b>Preface</b> | How I learned Python, and why I wrote this | 5  |
| <b>01</b>      | Why Python?                                | 6  |
| <b>02</b>      | Set up your tools                          | 9  |
| <b>03</b>      | The basics                                 | 12 |
| <b>04</b>      | Intermediate Python                        | 17 |
| <b>05</b>      | Advanced Python                            | 22 |
| <b>06</b>      | Where Python is used                       | 27 |
| <b>07</b>      | Specialise                                 | 32 |
| <b>08</b>      | Work like a professional                   | 34 |
| <b>09</b>      | Your 12-month plan                         | 36 |
| <b>10</b>      | Resources and glossary                     | 39 |
|                | About the author                           | 42 |

## PREFACE · UTANGULIZI

# How I learned Python.

I learned Python at Mbeya University of Science and Technology (MUST), during my Diploma in Computer Science. Class gave me the basics. The rest I learned on my own: free online resources, a lot of small projects, and more error messages than I can count.

Nobody handed me a map. I jumped between tutorials, got stuck, started again and slowly figured out what to learn next. Looking back, I could have saved months with a clear path: what to learn first, what to build at each step and where the language can take you.

Then one of my students asked me for exactly that: a guide from the very beginning to working like a professional. This book is my answer. It's the roadmap I wish I'd had on my first day.

Read it in order. Type every example. Build every project. Don't worry about being slow; worry only about stopping. Kidogo kidogo, utafika.

*Privatus Cosmas*

Mbeya, October 2026

# Why Python?

Lugha ya kila fani

01

Before you write a single line, it helps to know what you're learning, why it's worth it, and how this book will get you there.

## IN THIS CHAPTER

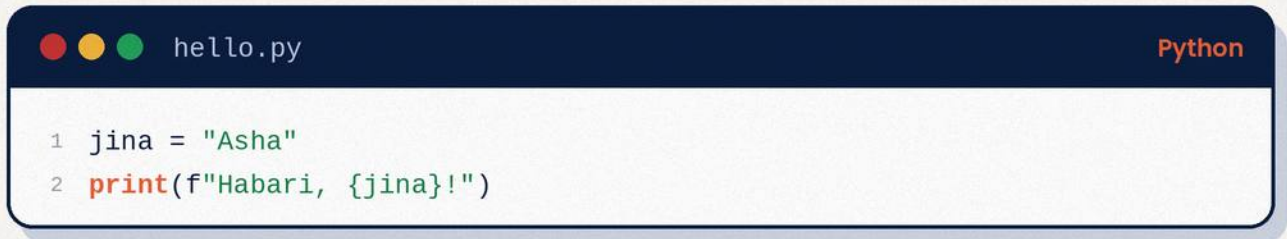
What Python is

Why it's worth learning

How to use this book

## What Python is

Python is a programming language: a way of writing instructions that a computer can follow. It was created by Guido van Rossum and first released in 1991. Its big idea is that code should be easy for humans to read. That is why Python looks almost like English, and why so many schools and universities teach it first.



```
1 jina = "Asha"
2 print(f"Habari, {jina}!")
```

Even if you have never programmed before, you can probably guess what those two lines do: store a name, then greet it.

## Why it's worth learning

- It reads almost like English, so you spend your energy on logic, not on symbols.
- One language opens many careers: web development, data, AI, automation, cybersecurity, DevOps, research and more.
- It has a huge, free community. Almost every error you will meet, someone has already met and explained online.
- It is used by companies large and small, from global platforms to the startup down the road.

## How to use this book

The book follows six levels. Each level has a goal, the topics to learn, real code, practice exercises and a project. Don't rush. Move to the next level when you can do the practice without looking everything up.

| Level   | Name         | Rough time |
|---------|--------------|------------|
| Level 0 | Set up       | Day 1      |
| Level 1 | Basics       | Weeks 1–6  |
| Level 2 | Intermediate | Months 2–3 |
| Level 3 | Advanced     | Months 4–6 |
| Level 4 | Specialise   | Months 6–9 |
| Level 5 | Professional | Month 9+   |

**ROUGH TIMES**

These assume one to two hours a day. Going slower is fine. Stopping is the only thing that doesn't work.

Type every example yourself. Don't copy and paste. Then change it, break it and fix it. That is how the knowledge moves from the page into your hands.

# Set up your tools

*Anza na ulicho nacho*

02

Level 0 takes one evening. By the end you will have Python installed, a proper code editor and your first program running.

## IN THIS CHAPTER

1. Install Python

2. Install VS Code

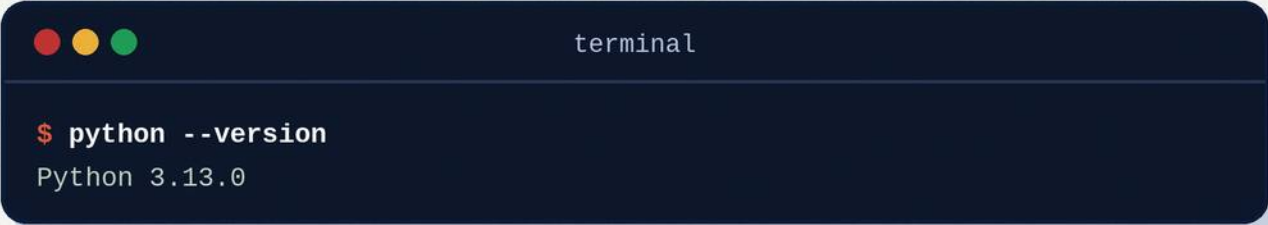
3. Run your first program

No laptop yet?

## 1. Install Python

Go to [python.org](https://python.org), open Downloads and install the latest Python 3 version for your system.

- Windows: run the installer and tick “Add python.exe to PATH” before you click Install. This one tick saves hours of confusion.
- macOS: install from [python.org](https://python.org), or with Homebrew if you already use it.
- Linux: Python 3 is usually already installed. Check in the terminal.



```
terminal  
  
$ python --version  
Python 3.13.0
```

### NOTE

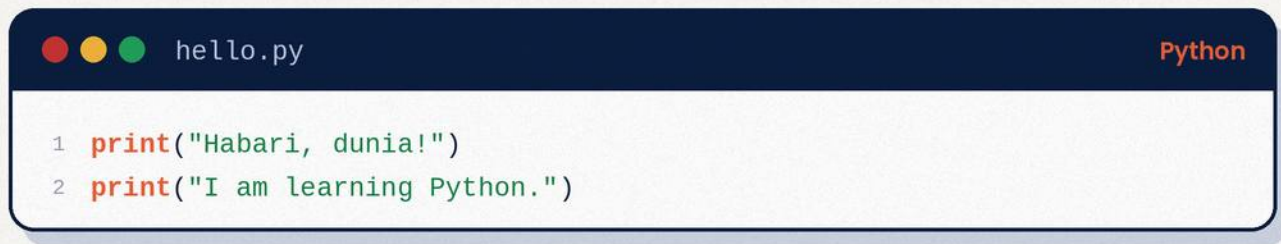
On some systems the command is `python3` instead of `python`. If one doesn't work, try the other.

## 2. Install VS Code

Download Visual Studio Code (free) from [code.visualstudio.com](https://code.visualstudio.com). Open it, go to Extensions and install the official “Python” extension by Microsoft. It gives you colours, suggestions, error hints and a Run button.

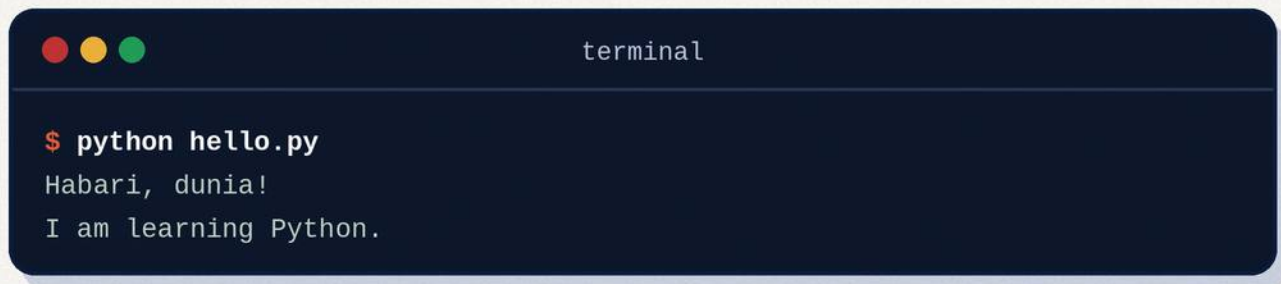
## 3. Run your first program

Make a folder called `python-journey`. Open it in VS Code, create a file called `hello.py` and type:



```
hello.py Python
1 print("Habari, dunia!")
2 print("I am learning Python.")
```

Save it, then run it from the terminal inside VS Code:



```
terminal
$ python hello.py
Habari, dunia!
I am learning Python.
```

## No laptop yet?

You can start in the browser with Google Colab or Replit, even on a phone. It is not ideal for big projects, but it is perfect for Level 1. Don't let missing equipment delay you.

### PRACTICE · MAZOEZI

- ☐ Install Python and check the version.
- ☐ Install VS Code and the Python extension.
- ☐ Run a file that prints your name and your town.

# The basics

*Msingi imara*

03

Level 1, weeks 1 to 6. These ideas exist in every programming language. Learn them well here and every language after Python becomes easier.

## IN THIS CHAPTER

Variables and data types

Strings and input

Conditions

Loops

Functions

Data structures

## Variables and data types

A variable is a name that points to a value. Python works out the type from the value you give it.

 types.py

Python

```
1 name = "Asha"           # str: text
2 age = 19                 # int: whole number
3 height = 1.62            # float: decimal
4 is_student = True        # bool: True or False
5
6 print(type(age))         # <class 'int'>
```

## Strings and input

Strings hold text. f-strings let you put variables inside text. `input()` reads what the user types, always as a string, so convert it when you need a number.

 greet.py

Python

```
1 name = input("Jina lako? ")
2 year = int(input("Mwaka uliozaliwa? "))
3 age = 2026 - year
4 print(f"{name.title()}, you are about {age}.")
```

## Conditions

`if`, `elif` and `else` let your program make decisions. Indentation (four spaces) shows which lines belong to which branch.

```
grade.py Python
1 mark = 72
2
3 if mark >= 75:
4     print("A")
5 elif mark >= 65:
6     print("B")
7 elif mark >= 45:
8     print("C")
9 else:
10    print("Fail")
```

## Loops

A for loop repeats over a sequence. A while loop repeats until a condition becomes false.

```
loops.py Python
1 marks = [78, 45, 90, 62]
2 for m in marks:
3     print(m, "Pass" if m >= 50 else "Fail")
4
5 count = 3
6 while count > 0:
7     print(count)
8     count -= 1
```

## Functions

A function is a named block of code you write once and use many times. It takes inputs (parameters) and can return a result.

functions.py

Python

```
1 def average(numbers):
2     return sum(numbers) / len(numbers)
3
4 print(average([78, 45, 90])) # 71.0
```

## Data structures

| Type  | Example          | Use it for                    |
|-------|------------------|-------------------------------|
| list  | [78, 90, 62]     | Ordered items you can change  |
| dict  | {"name": "Asha"} | Values found by a key         |
| tuple | (-6.8, 39.2)     | Fixed values, like a location |
| set   | {"Dar", "Mbeya"} | Unique items                  |

student.py

Python

```
1 student = {"name": "Asha", "marks": [78, 90, 62]}
2 student["marks"].append(85)
3 best = max(student["marks"])
4 print(f"{student['name']} best mark: {best}")
```

## Errors and files

Errors are normal. Read them from the bottom line up: it names the error and the line. Use try/except to handle errors you expect.

safe.py

Python

```
1 try:
2     age = int(input("Age? "))
3 except ValueError:
4     print("Please type a number.")
5
6 with open("notes.txt", "a") as file:
7     file.write("Today I learned loops.\n")
```

## Practice

PRACTICE · MAZOEZI

- ☐ Ask for two numbers and print their sum, difference and product.
- ☐ Print the multiplication table of any number the user types.
- ☐ Write a function `is_even(n)` that returns True or False.
- ☐ Count how many times each word appears in a sentence (use a dict).
- ☐ Save a list of names to a file, then read them back.

## Project: grade calculator

Ask for a student's name and five marks, then print the average, the highest mark and the grade, and save the result to a file. Handle wrong input without crashing. When you can build this without a tutorial, you're ready for Level 2.

# Intermediate Python

*Sasa unajenga*

# 04

Level 2, months 2 to 3. You'll stop writing everything yourself and start using other people's code, organise your own code into classes, and talk to the internet.

## IN THIS CHAPTER

Modules and the standard library

pip and virtual environments

Classes and objects

Comprehensions

JSON and CSV

Talking to APIs

## Modules and the standard library

Python comes with batteries included: hundreds of ready modules. Import what you need.

 dates.py

Python

```
1 from datetime import date
2 import random
3
4 print(date.today())
5 print(random.choice(["Dar", "Mbeya", "Arusha"]))
```

## pip and virtual environments

pip installs packages written by others. A virtual environment keeps each project's packages separate, so projects don't break each other.



terminal

```
$ python -m venv .venv
# Windows: .venv\Scripts\activate
$ source .venv/bin/activate
$ pip install requests
$ pip freeze > requirements.txt
```

## Classes and objects

A class is a blueprint; an object is a thing built from it. Classes group data and the functions that work on it.

 account.py

Python

```
1 class Account:
2     def __init__(self, owner, balance=0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount):
7         self.balance += amount
8
9 acc = Account("Asha")
10 acc.deposit(5000)
11 print(acc.owner, acc.balance)
```

## Comprehensions

 comprehension.py

Python

```
1 marks = [78, 45, 90, 62]
2 passed = [m for m in marks if m >= 50]
3 squares = {n: n * n for n in range(5)}
```

## JSON and CSV

Most real data arrives as JSON (from APIs) or CSV (from spreadsheets). Python reads both out of the box.

```
sales.py Python
1 import csv
2
3 with open("sales.csv") as file:
4     rows = list(csv.DictReader(file))
5
6 total = sum(float(r["amount"]) for r in rows)
7 print(f"Total sales: {total:,.0f}")
```

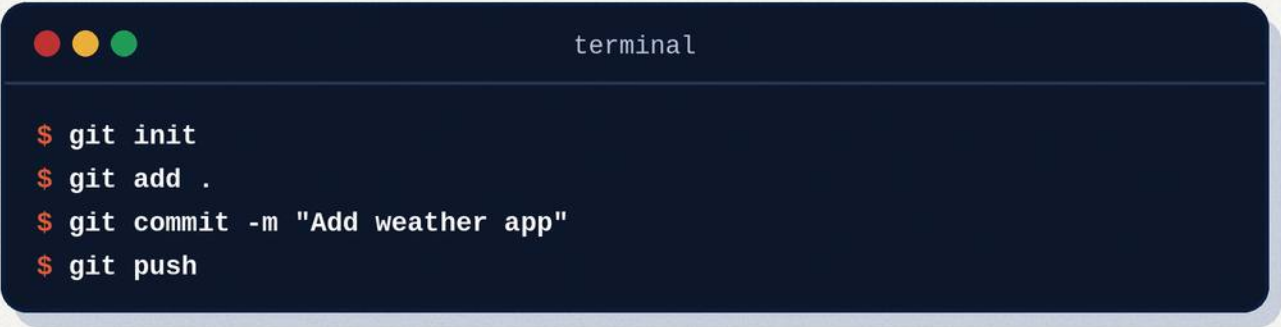
## Talking to APIs

An API is a door into another service. The requests package fetches data in a few lines. This uses the free Open-Meteo weather API (no key needed):

```
weather.py Python
1 import requests
2
3 url = "https://api.open-meteo.com/v1/forecast"
4 params = {"latitude": -8.9, "longitude": 33.46,
5           "current_weather": True}
6 data = requests.get(url, params=params).json()
7 print(data["current_weather"]["temperature"])
```

## Git and GitHub

Git saves the history of your code; GitHub stores it online and becomes your public portfolio. Learn these from now on, every day.

A terminal window with a dark blue background and light blue text. The title bar shows three colored circles (red, yellow, green) and the word "terminal". The terminal contains four lines of commands, each preceded by a red prompt character "\$".

```
$ git init
$ git add .
$ git commit -m "Add weather app"
$ git push
```

## Practice

### PRACTICE · MAZOEZI

- ☐ Create a class Student with a method that returns the average mark.
- ☐ Read a CSV of expenses and print the total per category.
- ☐ Fetch data from a free public API and print three facts from it.
- ☐ Put every project from now on in its own GitHub repository.

## Project: weather app

Ask for a town, look up its coordinates in a small dict, fetch the current weather and print a friendly message. Save every search to a JSON file and add a command that shows search history.

# Advanced Python

*Ubora ni kazi*

05

Level 3, months 4 to 6. Now you'll learn to write code that other people can read, trust and build on. This is the difference between code that works on your laptop and code a company can use.

## IN THIS CHAPTER

Type hints

Testing with pytest

Clean code

Decorators and generators

Databases with SQL

A first look at async

## Type hints

Type hints say what a function expects and returns. Python doesn't enforce them, but editors and tools use them to catch mistakes early.

 typed.py

Python

```
1 def average(marks: list[float]) -> float:
2     return sum(marks) / len(marks)
```

## Testing with pytest

A test is code that checks your code. Run pytest and it finds every function that starts with test\_.

 test\_marks.py

Python

```
1 from typed import average
2
3 def test_average():
4     assert average([50, 100]) == 75
5
6 def test_single_mark():
7     assert average([80]) == 80
```



terminal

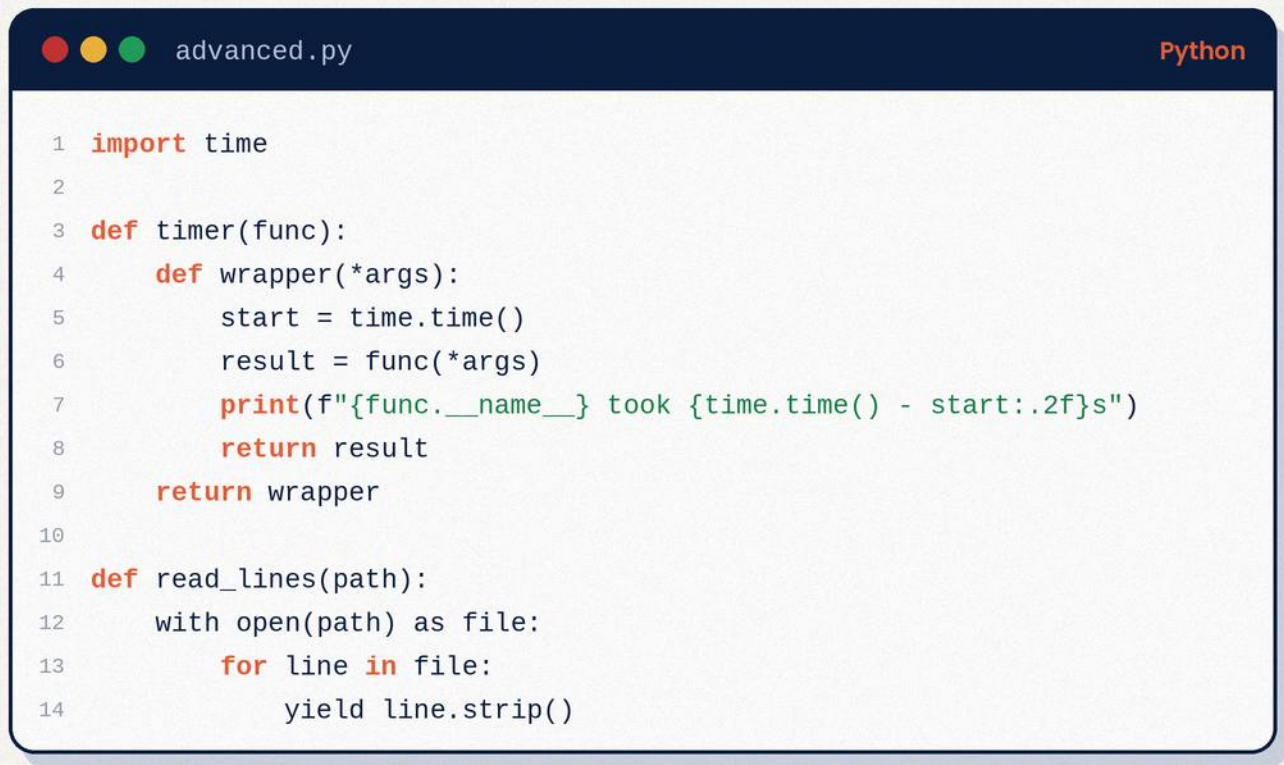
```
$ pip install pytest
$ pytest
2 passed in 0.01s
```

## Clean code

- Follow PEP 8, Python's style guide. Let a tool like ruff check and format your code automatically.
- Name things clearly: `total_sales`, not `ts`.
- Keep functions small: one function, one job.
- Write a README for every project: what it does, how to run it.

## Decorators and generators

A decorator wraps a function to add behaviour. A generator produces values one at a time with `yield`, which saves memory on big data.



```
1 import time
2
3 def timer(func):
4     def wrapper(*args):
5         start = time.time()
6         result = func(*args)
7         print(f"{func.__name__} took {time.time() - start:.2f}s")
8         return result
9     return wrapper
10
11 def read_lines(path):
12     with open(path) as file:
13         for line in file:
14             yield line.strip()
```

## Databases with SQL

Real apps store data in databases. Python includes `sqlite3`, so you can learn SQL with nothing to install.

```
db.py Python
1 import sqlite3
2
3 con = sqlite3.connect("shop.db")
4 con.execute("CREATE TABLE IF NOT EXISTS sales (item TEXT, amount REAL)")
5 con.execute("INSERT INTO sales VALUES (?, ?)", ("Sugar", 3500))
6 con.commit()
7 print(con.execute("SELECT SUM(amount) FROM sales").fetchone())
```

**SECURITY**

Always use ? placeholders for values, never f-strings inside SQL. It protects you from SQL injection.

## A first look at async

async lets one program wait for many slow things at once, such as many web requests. You'll meet it in FastAPI and bots. Understand the idea now; master it later in your specialisation.

## Practice

**PRACTICE · MAZOEZI**

- ☐ Add type hints to every function in your Level 2 projects.
- ☐ Write at least five pytest tests for your grade calculator.
- ☐ Run ruff on your code and fix what it reports.
- ☐ Store your weather history in SQLite instead of JSON.

## Project: expense tracker

A command-line app that adds, lists and deletes expenses stored in SQLite, shows totals per category and per month, and has tests for every calculation. Put it on GitHub with a proper README. This is a portfolio project.

# Where Python is used

*Python iko kila mahali*

Python isn't one job. It's a key that opens twenty different doors. This chapter walks through every major area.

## IN THIS CHAPTER

Web backends

Data analysis

Machine learning

AI apps & LLMs

Automation

Web scraping

Here is where Python is used today, with the libraries professionals use and a first project for each. Read them all, then pick the one that excites you most. That choice is Level 4.

**BIG NAMES**

Instagram, Spotify, Netflix, Dropbox and NASA have all used Python in their work.

**01 Web backends**

Websites, APIs, admin panels and logins

Django · FastAPI · Flask

**First project: A school results API**

**02 Data analysis**

Turn spreadsheets and records into answers

pandas · NumPy · Jupyter

**First project: Analyse a shop's sales CSV**

**03 Machine learning**

Models that predict, sort and recommend

scikit-learn · PyTorch · TensorFlow

**First project: Predict house prices**

**04 AI apps & LLMs**

Chatbots, assistants and AI features in apps

LLM APIs · Hugging Face · LangChain

**First project: A Swahili FAQ chatbot**

**05 Automation**

Scripts that do boring work for you

pathlib · openpyxl · schedule

**First project: Auto-build a weekly Excel report**

**06 Web scraping**

Collect data from websites

requests · BeautifulSoup · Scrapy

**First project: Track new job posts every day**

**07 Data visualisation**

Charts, reports and dashboards

Matplotlib · Plotly · Streamlit

**First project: A live sales dashboard**

**08 Computer vision**

Teach computers to see images and video

OpenCV · Ultralytics YOLO

**First project: Count cars at a junction**

**09 NLP & language**

Understand text and speech

spaCy · NLTK · Transformers

**First project: Sort customer messages by topic**

**10 Cybersecurity**

Security tools, testing and analysis

Scapy · Impacket · pwntools

**First project: A simple port scanner**

**11 DevOps & cloud**

Automate servers, deploys and the cloud

Ansible · boto3 · Fabric

**First project: Auto-backup files to the cloud**

## 12 Testing & QA

Check that software really works

pytest · Selenium · Playwright

**First project: Test a login page automatically**

## 13 Bots

Telegram, Discord and chat bots

python-telegram-bot · discord.py

**First project: A class reminder bot**

## 14 Desktop apps

Apps for Windows, Mac and Linux

Tkinter · PyQt

**First project: A shop stock manager**

## 15 Games

2D games and quick prototypes

Pygame · Arcade

**First project: A Snake game**

## 16 IoT & hardware

Raspberry Pi, sensors and robots

MicroPython · GPIO Zero

**First project: A temperature alarm**

## 17 Science & research

Biology, physics, health and space

SciPy · Biopython · Astropy

**First project: Analyse lab results**

**18 Finance & fintech**

Analysis, risk and reporting

pandas · NumPy · QuantLib

**First project: Track your M-Pesa spending**

**19 GIS & maps**

Maps and location data

GeoPandas · Folium

**First project: Map clinics in your region**

**20 Education**

The first language in many schools

Jupyter · turtle

**First project: Teach a friend to code**

# Specialise

*Njia moja kwanza*

07

Level 4, months 6 to 9. Pick one path from the last chapter and go deep.

IN THIS CHAPTER

By now you know Python itself. The next step is to go deep in one area. One path for six months beats six paths for one month.

## Backend

Learn in this order: Django or FastAPI, SQL, auth, deploy.

Serious project: A booking API with logins.

## Data analyst

Learn in this order: pandas, SQL, charts, storytelling.

Serious project: A sales report with insights.

## ML / AI

Learn in this order: NumPy, pandas, scikit-learn, PyTorch.

Serious project: A model that predicts prices.

## Automation

Learn in this order: files, Excel, web, scheduling.

Serious project: A daily report robot.

## Cybersecurity

Learn in this order: networking, sockets, Scapy, CTFs.

Serious project: A network scanner.

## DevOps

Learn in this order: Linux, scripts, Docker, CI/CD, cloud.

Serious project: An auto-deploy pipeline.

### CAN'T DECIDE?

Choose the area whose first project you would build even if nobody paid you.  
Curiosity carries you further than salary.

# Work like a professional

*Kuaminika ni ujuzi*

Level 5, month 9 and beyond. Professional doesn't mean knowing everything. It means being reliable: your code works, others can read it, and you deliver what you promise.

## IN THIS CHAPTER

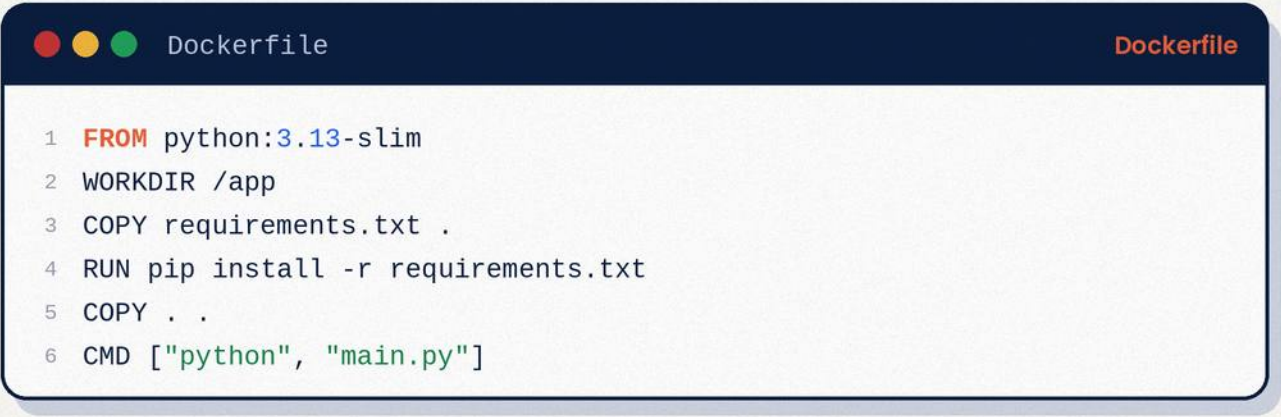
The professional toolkit

Getting hired

Keep growing

## The professional toolkit

- Git every day: branches, pull requests and clear commit messages.
- Tests and code reviews: give feedback and receive it without taking it personally.
- Docker: package your app so it runs the same on every machine.
- Deploys: your code runs on servers, not only on your laptop.
- Documentation: READMEs and comments that help the next person.



```
1 FROM python:3.13-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install -r requirements.txt
5 COPY . .
6 CMD ["python", "main.py"]
```

## Getting hired

- Three strong portfolio projects, live and explained.
- A clean GitHub and LinkedIn.
- Practice problem solving: lists, dicts, strings, sorting, searching and recursion, on sites like LeetCode or HackerRank.
- Real experience: freelance jobs, open source contributions, hackathons or a team project.

## Keep growing

Read other people's code. Follow the release notes of the tools you use. Teach what you learn; explaining is the fastest way to understand. And keep building: the habit that got you here is the habit that will keep you here.

# Your 12-month plan

*Polepole ndio mwendo*

Everything in this book, on one calendar.

## IN THIS CHAPTER

A daily routine

Habits that get you there

Mistakes to avoid

| Months | Focus             | Finish with              |
|--------|-------------------|--------------------------|
| 1      | Set up + basics   | Small programs           |
| 2      | Basics + practice | Grade calculator         |
| 3      | Intermediate      | Weather app              |
| 4–5    | Advanced          | Expense tracker          |
| 6–9    | Specialise        | One serious project      |
| 10–12  | Professional      | Portfolio + applications |

## A daily routine

### 01 10 minutes

Review yesterday's code.

### 02 40 minutes

Learn one new thing and type the examples.

### 03 30 minutes

Build: add one small feature to your project.

### 04 5 minutes

Commit and push to GitHub.

## Habits that get you there

- Code a little every day. One hour daily beats a weekend marathon.
- Read the error message, bottom line first.
- Build before you feel ready. You never will; build anyway.
- Use AI as a tutor: ask it to explain, not to do your work.
- Learn in public: share progress on LinkedIn or your status.

## **Mistakes to avoid**

- Jumping to Django or AI before the basics.
- Watching tutorials without building anything.
- Switching languages every few weeks.
- Comparing your chapter 1 to someone else's chapter 10.

# Resources and glossary

Pakua, soma, jenga

10

Free places to keep learning, and the words you'll hear most.

## IN THIS CHAPTER

Free resources

Glossary

---

## Free resources

- 01 CS50's Python course (CS50P)**  
Harvard's free course, with problem sets
- 02 The python.org tutorial**  
The official, free tutorial
- 03 Automate the Boring Stuff**  
Free to read online; perfect for automation
- 04 Exercism Python track**  
Free exercises with mentor feedback
- 05 Kaggle Learn**  
Free short courses on pandas and ML
- 06 Real Python**  
Clear tutorials on almost every topic
- 07 freeCodeCamp**  
Free Python and data courses with certificates
- 08 Official docs**  
docs.python.org: the final word on everything

## Glossary

### 01 Variable

A name that points to a value.

### 02 Function

A named, reusable block of code.

### 03 Module

A file of Python code you can import.

### 04 Package

A collection of modules, often installed with pip.

### 05 Library / framework

Ready code for common jobs; a framework also sets the structure.

### 06 API

A way for programs to talk to each other.

### 07 Virtual environment

A separate space for one project's packages.

### 08 Bug

A mistake in code. Debugging is finding and fixing it.

### 09 Repository

A project folder tracked by Git.

### 10 Deploy

Putting your app on a server so others can use it.

## ABOUT THE AUTHOR

# Privatus Cosmas

## Software Engineer & Digital Innovator

Privatus Cosmas, known online as depriver, is from Kitwe village in Kyerwa District, Kagera, Tanzania. He studied a Diploma in Computer Science at Mbeya University of Science and Technology (MUST).

He works as a Software Engineer and Digital Innovator at Rohi Company Limited, and shares practical tech lessons for students in Swahili and English at [depriver.tech](https://depriver.tech).

## SAY HELLO · KARIBU

|                  |                                                                                                  |
|------------------|--------------------------------------------------------------------------------------------------|
| Website          | <b><a href="https://depriver.tech">depriver.tech</a></b>                                         |
| Instagram & more | <b><a href="#">@_depriver</a></b>                                                                |
| This book online | <b><a href="https://depriver.tech/blog/python-roadmap">depriver.tech/blog/python-roadmap</a></b> |

# Start today.

## Finish what you start.

*Maliza unachoanza*

|                        |            |
|------------------------|------------|
| Level 0 · Set up       | Day 1      |
| Level 1 · Basics       | Weeks 1–6  |
| Level 2 · Intermediate | Months 2–3 |
| Level 3 · Advanced     | Months 4–6 |
| Level 4 · Specialise   | Months 6–9 |
| Level 5 · Professional | Month 9+   |



**Privatus Cosmas**

depriver.tech · @\_depriver